

接受之后：基于 eBPF 校准的残差机器论断图谱

独立研究者·emtanling@gmail.com

Chengao Zhang

版本说明 (2026-07-13): 本文件是供作者核对论证的中文阅读版, 不是逐句翻译, 也不是投稿规范源。唯一投稿规范源是英文 PAPER_REPORT.tex 及其编译 PDF; 如中文表述与英文规范源不一致, 以英文为准。

摘要

语言理论安全 (language-theoretic security) 关注的是: 一个边界所识别的语言, 是否就是下游机制实际解释的语言。在程序验证器边界, 已接受制品可能驱动某些有状态的运行时操作, 而这些操作并未由预期的报告关系描述。我们引入一个包含五个节点的论断图谱, 将制品接受 (A)、同后缀因果状态区分 (C)、有界可编程性 (P)、已计算报告不可因子分解性 (R) 以及策略/威胁义务 (W) 区分开来。未来观测商集 (future-observation quotient) 与行为因子分解判据, 将以已接受制品为索引的因果语言同相对于报告的残差语言 (residual language) 区分开来。该判据只相对于一个具体报告实例, 并且只在选定上下文纤维上的**唯一单元报告映射**成立时适用; 本文不声称每个安全健全但不完备的验证器都必然承载怪异机器。

我们给出如下证明义务: 一个受到统一控制、可观测且可重置的门, 以及一个在有界电路描述域上固定不变的已接受解释器。在精确调度、可接纳性、串行化与帧保持这些前提下, 前缀归纳可推出有界组合性。在一项 Linux eBPF 标定中, 一个专用的、预分配的、非 LRU 两项哈希映射实现了 NAND: 重置后保留一个哨兵项; 输入比特决定是更新已有键还是新键; 第二次更新的成功谓词提供输出。一个固定不变且已接受的程序处理规范化的 NAND 有向无环图 (DAG), 其输入不超过 64 个、门不超过 512 个、活跃导线不超过 578 条。

一次带源码快照的 Linux/aarch64 运行覆盖了具名电路和随机电路、联合边界、串行复用、机制对照以及畸形描述符。一个由作者另行运行的语义审计器重建描述符与电路语义, 另有一份自行签发的清单检查证据包的完整性。该案例记录 A; 在声明的具体映射服务与无干扰契约下给出**条件性 C** 见证; 并在更多实现前提下支持 P。它没有确立 Linux 报告不可因子分解性, 也没有确立策略层面的怪异机器 (weird machine)。另有一个固定辅助可执行实例针对自身计算出的报告建立 $R(M_{linux_r_aux_v1})$; 该结果不改变 stock Linux 的状态。

关键词: 语言理论安全, 识别器(recognizer), 残差语言, 怪异机器, 程序验证, 抽象解释(abstract interpretation), eBPF。

1. 引言

语言理论安全将输入边界视为语言识别边界: 下游处理就是解释, 而输入提供被解释的程序 [1]–[4]。因此, 输入校验 (validation) 与程序验证 (program verification) 共同承担一种形如识别器的角色。我们的问题从接受之后开始。验证器接受一个程序制品, 但这个已接受程序仍可驱动一种由有状态辅助函数与服务操作构成的语言。若要将这种下游语言称为残差的、可编程的、由形状诱导的或怪异机器, 需要满足哪些论断?

制品语言与接受之后的操作语言具有不同的载体。验证器可以识别有界执行、带类型的辅助函数使用与内存访问

规约，却不必认证通过每一项已文档化运行时服务实现的完整关系。已接受程序可以选择操作、保留服务状态、观测返回、重置状态并组合效果。这些事实本身都不蕴含验证器缺陷。它们首先确立的是一种以下游操作为内容、以已接受制品为索引的语言。

本文将证据负担组织为如下论断图谱：

节点	必需事实	证据状态
A — 接受	一个固定制品属于 L_V	已针对保留的目标文件与环境记录
C — 因果状态区分	相同后缀与相同非状态上下文，从不同选定运行时状态暴露出不同观测	仅在声明的映射服务/无干扰契约下给出条件性见证
P — 有界可编程性	在明确的帧条件与环境前提下，已接受代码统一地控制、观测、重置并组合状态	源码级构造加经审计的回归证据
R — 相对于报告的残差	C 中的见证由一个实际的已计算报告单元共同覆盖，且该单元未通过行为因子分解测试	固定辅助元组：已建立；stock Linux：未建立
W — 策略/威胁义务	行为主体通过 P_* 驱动预期策略所排除的效果	离线案例尚未确立

上表的 A、C、P 与 W 状态报告 stock eBPF/Linux 案例。R 行明确区分两个载体：固定元组 $M_{linux_r_aux_v1}$ 针对辅助报告生成识别器和受限服务语义建立 R，而 stock Linux 验证器的 R 仍未建立。

节点 C 与 R 不得共用同一个名称。我们将以已接受制品为索引的同后缀语言称为 L_{causal} 。只有存在实际的已计算单元碰撞时，一个词才进入相对于报告的残差语言 L_{res}^R 。P 与 R 在 C 之后分叉：可编程性不蕴含报告碰撞，报告碰撞也不蕴含可编程性。策略层面的怪异机器还要求 P、W、同一编码计算上的链接 (Link) 与非预期性 (Unint)；一个更窄的由契约形状诱导、相对于识别器的分类还要求 R、已文档化且安全保持的语义 (Doc)、报告实现符合声明抽象 (Conf) 以及碰撞确由声明粒度强制 (Gran)。该图谱把“怪异机器”从一种修辞标签转化为可检查的义务。

可编程性是一个独立的构造问题。状态区分可能是无效的、不受控制的、一次性的，或无法组合的。因此，我们要求程序可见的读出、一个统一的输入分派器、向规范类的重置，以及一个固定不变的已接受解释器；该解释器必须在独立声明的有界描述符域上满足精确调度与帧保持。由此得到的组合定理是一种证明义务分解，而不是普遍的必然性定理。

我们的标定案例是 Linux eBPF。一个固定制品使用专用的两项哈希映射作为饱和资源。重置后，一个哨兵保持活跃。零比特更新该哨兵；一比特插入一个与输入对应的新键。第二个由输入决定的更新仅在输入为 (1, 1) 时失败，从而得到 NAND。普通字节码仍负责验证描述符、选择键、路由导线、控制循环并存储输出。宿主解析器把文本 WMC1 规范化到映射中，而验证器仅接受这个固定的 eBPF 制品。

该案例有意用于诊断。它记录 A，在声明的服务契约下给出条件性 C，并在所述实现前提下支持 P，尽管相同函数很容易用普通字节码表示。它还表明，为何不能在没有说明的情况下把 P 提升为 R，也不能把 P 与缺失的 W 义务合并。所保留的验证器日志不是已计算抽象单元的提取器，而离线运行没有提供任何遭违反的部署策略。

本文贡献如下：

- 论断图谱与带类型的语言。**我们区分已接受制品、因果运行时词、相对于报告的残差词、有界可编程性以及策略/威胁义务。未来观测商集与因子分解判据使节点 R 可测试，而反模型表明这些分支不会坍塌为一体。
- 有界组合论证。**E1–E3 与精确定义的 E4–D 解释器义务，将局部门正确性、重置、调度以及全局帧条件隔离开来。前缀归纳证明功能性结果；安全性是另一个可选前提。
- eBPF 标定案例。**一个饱和秩 NAND 基与固定解释器以 $D_{64,512}$ 为目标。所记录的测试套件覆盖具名 DAG 与使用固定种子的随机 DAG、深度边界与联合边界、零门案例、串行复用、机制对照以及畸形描述符。

4. **可执行报告实例。**一个固定辅助识别器/服务元组发出实际报告、构造有限未来观测商，并独立检查报告单元碰撞和四个负对照。它只针对该具名元组建立 R ，保持 stock Linux 边界不变。

实证范围限于一个具备特权的离线 Linux/aarch64 环境。该案例不是 Linux 报告不透明性结果、并发部署结果、制品参数化编译器、无界机器、漏洞或策略层面的怪异机器。

2. 相对于识别器的残差语言

2.1 识别、执行与报告

设 V 为程序制品上的识别器， I 为状态载体 Σ_I 与轨迹载体 $\mathcal{T}_I$ 上的具体执行语义，其中包括指令引擎、辅助函数、有状态服务以及相关环境；因此 $\text{Tr}_I(P) \subseteq \mathcal{T}_I$ 。已接受制品语言为

$$L_V = \{P \mid V(P) = \text{accept}\}.$$

对于可选的允许具体轨迹集合 $\text{Safe} \subseteq \mathcal{T}_I$ ，当下式成立时，该边界在安全性上是健全的：

$$\forall P \in L_V. \text{Tr}_I(P) \subseteq \text{Safe}.$$

安全健全性是一项前提，不能从一次成功加载中推断得出。它也不同于抽象变换器的完备性。抽象解释通过抽象映射与具体化映射关联具体集合和抽象元素 [5]，但验证器的已接受语言、它所计算的报告、传递函数的健全性及其完备性属于不同判断。尤其是，单元素集合的最佳抽象未必是分析器在汇合控制前沿实际计算的单元。

当报告属于讨论范围时，令 $\text{Report}_V(P, \ell) \subseteq A_\ell$ 表示在前沿 ℓ 实际计算的有限抽象单元集合，并为其配备已声明的具体化函数 $\gamma_\ell : A_\ell \rightarrow \mathcal{P}(\Sigma_I)$ 。前沿覆盖要求

$$\text{Reach}_I(P, \ell) \subseteq \bigcup_{a^\# \in \text{Report}_V(P, \ell)} \gamma_\ell(a^\#).$$

只有当一个已计算单元同时包含两个具体状态时，二者才受到共同覆盖。这一点排除了一种常见但无效的捷径：证明两个值具有相似的日志打印文本，并不能确定一个已计算抽象单元、一项具体化或共同覆盖。

2.2 因果词

对于 $P \in L_V$ 与前沿 ℓ ，令 $\Sigma_{\text{op}}(P)$ 为程序可控制运行时操作的有限字母表，并令 $W_{\text{run}}^\ell(P) \subseteq \Sigma_{\text{op}}(P)^*$ 为已接受代码能够从 ℓ 开始执行的词。若将所有此类词都称为残差，就只是为普通轨迹语义改了一个名字；因此，节点 C 使用同后缀因果测试，暂不主张报告有所遗漏。

在比较前固定观测契约

$$K_{\text{obs}} = (\rho_{\text{obs}}, \text{Obs}, \text{Slice}, \text{Env}).$$

其中 $\rho_{\text{obs}} : \Sigma_I \rightarrow R_{\text{obs}}$ 选取候选状态， $\text{Obs} : \mathcal{T}_I \rightarrow O_{\text{obs}}$ 投影完整具体轨迹， Slice 为每个词 w 指定带类型的上下文投影 $\text{ctx}_w : \Sigma_I \rightarrow C_w$ ，而 Env 是声明的固定环境实例集合。上下文保守地包含后缀或观察者读取的每个非选定分量；环境实例固定资源配置、调度、非确定性以及外部干扰选择。

写 $\llbracket w \rrbracket_{I,e}(\sigma) = (\tau, \sigma')$ 。契约在 $X \subseteq \Sigma_I$ 上对 w 健全，当任意 $e \in \text{Env}$ 与 $\sigma, \sigma' \in X$ 若满足

$$\rho_{\text{obs}}(\sigma) = \rho_{\text{obs}}(\sigma'), \quad \text{ctx}_w(\sigma) = \text{ctx}_w(\sigma'),$$

则两次后缀执行具有相同的有定义性；若均有定义，则完整轨迹投影相同。也就是说， $(\rho_{\text{obs}}, \text{ctx}_w, e)$ 必须包含所有可影响声明观测的因素。缺少这一非干扰条件时，不接纳 C 见证。

定义 1 (因果状态介导词族)。 当 $P \in L_V$ 、 $w \in W_{\text{run}}^\ell(P)$ ， K_{obs} 在 $\text{Reach}_I(P, \ell)$ 上对 w 健全，且存在同一个 $e \in \text{Env}$ 与 $\sigma_0, \sigma_1 \in \text{Reach}_I(P, \ell)$ ，使得 $\llbracket w \rrbracket_{I,e}(\sigma_i) = (\tau_i, \sigma'_i)$ 均有定义并终止且满足下列条件时，称词 w 在 (P, ℓ) 处是因果的：

$$\begin{aligned} \text{ctx}_w(\sigma_0) &= \text{ctx}_w(\sigma_1), \\ \rho_{\text{obs}}(\sigma_0) &\neq \rho_{\text{obs}}(\sigma_1), \\ \text{Obs}(\tau_0) &\neq \text{Obs}(\tau_1). \end{aligned}$$

定义依赖类型的带标签词族

$$L_{\text{causal}}(V, I; K_{\text{obs}}) = \{(P, \ell, w) \mid w \text{ 在 } (P, \ell) \text{ 处是因果的}\}.$$

制品与前沿标签是该类型的一部分。为制品、前沿与操作符号固定可解码的前缀编码后，该词族可以嵌入一个有限字母表上的普通语言；依赖类型记法则使各载体保持可见。宿主描述符既不是 L_V 的另一个元素，也不会自动成为运行时间。它首先成为配置，已接受解释器据此导出调度，而只有同后缀见证才进入 L_{causal} 。

该定义相对于观测者与切片。看到结果之后再从上下文中删除较早的输入，会使测试陷入循环论证。因此，eBPF 案例在比较之前就从源码语义声明其后缀；保留转换后的指令转储供人工检查，而不把它交给自动化切片检查器处理。

2.3 行为商集与相对于报告的残差语言

固定一个确定性规约

$$D = (X_D, S_D, A_D, O_D, \delta_D, \lambda_D, s_D, \text{Obs}_D)$$

其中 $X_D \subseteq \Sigma_I$ 是可接纳具体区域， S_D 是状态载体， A_D 是操作字母表， O_D 是输出字母表，且 $\text{Obs}_D : O_D^* \rightarrow O_{\text{obs}}$ 。转移和输出是同定义域偏函数

$$\delta_D : S_D \times A_D \rightarrow S_D, \quad \lambda_D : S_D \times A_D \rightarrow O_D.$$

每当 D 用于 (P, ℓ) 时，固定单射操作编码 $\iota_{P,\ell} : A_D \rightarrow \Sigma_{\text{op}}(P)$ 并同态扩展到词。当且仅当对每个 $\sigma \in X_D$ 与 $a \in A_D$ ，具体编码步骤存在恰好对应 $\delta_D(s_D(\sigma), a)$ 有定义，并且只要 $\sigma \xrightarrow{\iota_{P,\ell}(a)/o} \sigma'$ ，就有

$$\lambda_D(s_D(\sigma), a) = o, \quad s_D(\sigma') = \delta_D(s_D(\sigma), a).$$

并且 $\sigma' \in X_D$ ，称 $(s_D, \iota_{P,\ell})$ 在 X_D 上**操作充分**。这里 o 是该编码操作的完整声明观测。因此编码是语义重命名而不是任意映射：一个投影状态不能隐藏不同的有定义性、完整输出或后继投影。影响转移的每一种环境选择都由 D 固定，或包含在 S_D 中。

令 $\text{Out}_D(r, w)$ 表示通过迭代 (δ_D, λ_D) 得到的、可能无定义但一旦有定义便完整的输出词，以 $\text{Def}_D(r, w)$ 表示 $\text{Out}_D(r, w) \downarrow$ ，并取延续全集为 $\mathcal{W}_D = A_D^*$ ；不可行词以输出无定义表示。对于 $r, r' \in S_D$ ，定义

$$\begin{aligned}
r \sim_D r' &\iff \forall w \in \mathcal{W}_D. \\
&\left([\text{Def}_D(r, w) \iff \text{Def}_D(r', w)] \wedge \right. \\
&\quad \left. [\text{Def}_D(r, w) \Rightarrow \text{Out}_D(r, w) = \text{Out}_D(r', w)] \right).
\end{aligned}$$

由于 $\mathcal{W}_D = A_D^*$ 在操作左前缀下闭合, $\sim_D$ 是右同余: 有定义且匹配的 a 步将导向等价后继状态, 因为每个后继延续 w 都会作为 aw 被测试。若 $Q_D = S_D / \sim_D$ 有限, 限制在 X_D 的具体执行会在商状态上导出一个偏 Mealy 转导器。这延续 Nerode 与 Mealy 的继续等价传统, 并与观测完备性和广义强保持相关 [6]–[9]; 本文只测试一个已接受制品的一个前沿上的实际单元, 而不是提出新的通用完备性理论。

对于具体集合 F , 定义共同启用的延续

$$\mathcal{W}_D(F) = \{w \in \mathcal{W}_D \mid \forall \sigma \in F. \text{Out}_D(s_D(\sigma), w) \downarrow\}.$$

命题 1 (上下文纤维上的行为因子分解)。固定已接受的 P 、前沿 ℓ 、规约 D 、观测契约 K_{obs} , 以及非空 $F \subseteq \text{Reach}_I(P, \ell) \cap X_D$ 。要求

$$\iota_{P, \ell}(\mathcal{W}_D(F)) \subseteq W_{\text{run}}^\ell(P),$$

观测契约对所有编码共同延续健全, 且 F 上所有对应上下文投影相同。还要求**观察者兼容性**: 若编码词 w 从 $\sigma \in F$ 执行产生具体轨迹 τ , 则右侧有定义且

$$\text{Obs}(\tau) = \text{Obs}_D(\text{Out}_D(s_D(\sigma), w)).$$

定义 $\beta_D(\sigma) = [s_D(\sigma)]_D$ 与 $F_{a^\#} = F \cap \gamma_\ell(a^\#)$ 。最后要求**唯一单元条件**

$$\forall \sigma \in F. \exists! a^\# \in \text{Report}_V(P, \ell). \sigma \in \gamma_\ell(a^\#),$$

并令 $\pi_R : F \rightarrow \text{Report}_V(P, \ell)$ 把每个状态映射到其唯一实际计算单元。则报告在 F 上因子分解未来观测商,

$$\exists h : \pi_R(F) \rightarrow Q_D. \beta_D = h \circ \pi_R,$$

当且仅当

$$\forall a^\# \in \pi_R(F). |\beta_D(F_{a^\#})| \leq 1.$$

因此, 报告不可因子分解等价于一个唯一实际计算单元包含两个不同 $\sim_D$ 类的状态。正向由复合函数立即得到; 反向中, 每个非空单元的唯一商类定义 h 。唯一单元条件使 π_R 真正成为函数; 报告标签重叠时必须另行声明成员签名映射, 不能直接套用本命题。□

记 $\text{Adm}(P, \ell, D, F; K_{\text{obs}})$ 表示下列条件全部成立: $P \in L_V$; $(s_D, \iota_{P, \ell})$ 在 X_D 上操作充分; F 是上述可迭非空纤维; 以及运行词包含、观察者兼容、健全观测契约、共同上下文和唯一单元条件。

定义 2 (相对于报告的输出见证残差语言)。固定一个可接纳元组。当满足下列条件时, 依赖标签词 $(P, \ell, D, F, a^\#, w)$ 是输出见证的报告相对残差:

$$\begin{aligned}
(P, \ell, \iota_{P, \ell}(w)) &\in L_{\text{causal}}(V, I; K_{\text{obs}}), \quad w \in \mathcal{W}_D(F), \\
\sigma_0, \sigma_1 &\in F \text{ 是编码词的定义 1 见证,} \\
\pi_R(\sigma_0) &= \pi_R(\sigma_1) = a^\#, \quad \beta_D(\sigma_0) \neq \beta_D(\sigma_1).
\end{aligned}$$

$L_{\text{res}}^R(V, I, \text{Report}; K_{\text{obs}})$ 是在所有满足 Adm 的元组上这些带标签词的并集。操作充分与观察者兼容把不同具体观测绑定到不同商类。商关系还区分有定义性，因此报告不可因子分解可能没有一个两边都终止并输出不同值的词；所以 L_{res}^R 只是因子分解失败的**输出见证子集**，并非所有失败的逆向刻画。它是前沿碰撞，不是声称完整全程序报告永远无法恢复最终函数。

实例化定义 2 需要实际已计算单元的提取器、已声明的具体化、覆盖以及一个固定的上下文纤维。保留的 Linux 日志没有提供这些对象，因此 eBPF 案例只在声明的服务契约下给出条件性 L_{causal} 见证，未确立 L_{res}^R 。

2.4 形状、缺陷与策略

由缺陷诱导的缝隙违反已声明的识别器、报告或运行时契约。一个使用已文档化且安全保持语义、并在报告意图认证的关系上具有 L_{res}^R 见证的案例，只是契约形状缝隙的证据；仍须以报告符合性和粒度证据排除错误变换器、汇合或提取器。策略层面的怪异机器还需要同一计算上的行为主体控制、策略排除效果与非预期解释。因此，普通有状态 API 可以实现转导器，却未必满足任一分类。

3. 有界状态介导的电路实现

3.1 从一种区分到可复用门

一个因果词可以揭示一种区分，却未必支持重复计算。固定一个确定性的门规约 D 、一个规范重置类 $q_0 \in Q_D$ （将其等同于 S_D 中相应的子集），以及一个非空的可容许具体状态集 $\text{Adm}_G \subseteq X_D$ 。门基 $(P_G, \text{reset}, G, \text{observe}, D, \text{Adm}_G)$ 的预声明操作编码在 X_D 上操作充分，并满足：

E1 — 因果基与观测。 存在输入 x_0, x_1 、前缀 p_0, p_1 、同一剩余后缀 w 与内部前沿 ℓ_G ，使 $u_G(x_i) = p_i w$ 。从规范重置类执行前缀后所达的两个状态，是定义 1 对编码后公共后缀的见证；完整门读出恰等于该因果观测，并且就是 E4-D 写入的门结果，而不是无关探针。

E2 — 统一输入控制。 被接受的 P_G 中有一个统一的分派器 G ，读取运行时比特 $x \in \{0, 1\}^2$ 并选择完整的门词 $u_G(x)$ 。对于每个满足 $s_D(\sigma) \in q_0$ 的 $\sigma \in \text{Adm}_G$ ，由 D 定义的执行均终止，并对函数 $g : \{0, 1\}^2 \rightarrow \{0, 1\}$ 产生相同的比特 $g(x)$ ，即

$$\text{observe}(\text{Out}_D(s_D(\sigma), u_G(x))) = g(x).$$

实验者并非一个针对每个输入选择不同常量程序的外部预言机。

E3 — 重置。 重置词满足 $\text{reset} \in A_D^*$ 。对于每个 $\sigma \in \text{Adm}_G$ ，它均有定义、保持声明的导线单元不变，并终止于满足 $s_D(\tau) \in q_0$ 的 $\tau \in \text{Adm}_G$ 。如果 Adm_G 中的两个重置前状态在这些导线单元及固定上下文上相同，并分别重置到 τ_0, τ_1 ，那么对于每个 $x \in \{0, 1\}^2$ ，

$$\text{Out}_D(s_D(\tau_0), u_G(x)) = \text{Out}_D(s_D(\tau_1), u_G(x)).$$

没有 E1，隐藏差异就不会产生程序可见的效果。没有 E2，实验者可能自行提供真值表。没有 E3，该通道可能只能使用一次。下面的第四个条件在不把电路语义置于外部预言机中的前提下组合该门。

3.2 描述符域

本制品使用有界域 $D_{64,512}$ 。一个描述符为

$$d = (m, n, (s_i^0, s_i^1)_{0 \leq i < n}),$$

其中 $0 \leq m \leq 64$ 、 $0 \leq n \leq 512$ ，并且对于每个门 i 和操作数 b ,

$$0 \leq s_i^b < 2 + m + i$$

记 $m(d) = m$ 、 $n(d) = n$ 。导线 0 是常量零，导线 1 是常量一，主输入占用导线 2 至 $2 + m - 1$ ，门 i 写入规范目标 $2 + m + i$ 。因此，存活的规范导线数最大为 $2 + 64 + 512 = 578$ ，最高有效导线索引为 577。

对于 $x \in \{0, 1\}^{m(d)}$ 和门函数 g ， $\text{Eval}_g(d, x)$ 初始化常量和输入向量，随后按照描述符顺序扩展导线向量：

$$\nu[2 + m + i] = g(\nu[s_i^0], \nu[s_i^1]).$$

宿主编码 $\text{Enc}_U(d, x)$ 将规范化描述符、输入和控制记录写入映射。文本格式 WMC1 是一种宿主序列化。WMC1 和 d 都不是由 V 接受的 BPF 程序； $\text{Sched}_U(d, x)$ 是被接受的 P_U 读取该编码时诱导出的操作调度。

对于 $c = \text{Enc}_U(d, x)$ ，令 $\text{PhysRun}_U(c) = (s, k, \mu)$ 包含最终状态码、已完成迭代次数和物理映射状态。定义相对于描述符的规范观测

$$\text{WireObs}_d(\mu) = (\mu[0], \dots, \mu[1 + m(d) + n(d)])$$

以及状态掩蔽接口

$$\text{Run}_{U,d}(c) = \begin{cases} (s, \text{WireObs}_d(\mu)), & s = \text{OK}, \\ (s, \perp), & s \neq \text{OK}. \end{cases}$$

宿主只能从 OK 结果中投影所请求的输出导线。这条语义规则并未断言过期的物理单元已被擦除。

E4-D — 有界数据参数化解释。 当下列条件成立时，一个固定制品 P_U 在 $D_{64,512}$ 上履行 E4-D；其门基的制品分量必须是同一个 P_U ，而不是另一个门程序：

1. 对于固定的映射定义和所记录的加载环境， $P_U \in L_V$ ，且这一点与描述符内容 d 和 x 无关；
2. 每个有效的 $\text{Enc}_U(d, x)$ 都建立一个回调前状态 $\mu^{(0)}$ ，其中 $\mu^{(0)}[0] = 0$ 、 $\mu^{(0)}[1] = 1$ ，且对于 $0 \leq j < m(d)$ 有 $\mu^{(0)}[2 + j] = x_j$ ；它严格按顺序执行回调位置 $0, \dots, n(d) - 1$ ，并以 $\text{PhysRun}_U = (\text{OK}, n(d), \mu)$ 终止；
3. 一个外部临界区覆盖足迹中每个共享映射的设置、调用和读回；
4. 每次迭代 i 都验证规范形式，并恰好复制其两个较早的源导线；紧接重置之前的完整具体状态 σ_i 属于 Adm_G ；随后，该次迭代进行重置并调用 E1–E3 门，仅将得到的 g 比特写入目标 $2 + m(d) + i$ 和声明的审计单元，并保持描述符及所有较早导线不变；以及
5. 任何执行都不超过 512 次迭代；与此同时，由所声明的描述符校验器覆盖的格式错误核心 ABI 控制或描述符配置以非 OK 状态终止，完成的迭代次数至多为 512，且语义结果为 $(s, \perp)$ 。

这些条款是有意设定的证明义务。特别是，E4-D 要求写入门结果，并满足精确的迭代与终止行为；仅仅遍历描述符并不足够。

3.3 实现定理

定理 1 (显式义务下的有界组合)。假设固定的 P_U 在其声明的确定性且串行化环境下履行 E4-D, 并且其嵌入的门基以函数 g 满足 E1–E3。那么, 对于每个 $d \in D_{64,512}$ 和 $x \in \{0,1\}^{m(d)}$,

$$\text{Run}_{U,d}(\text{Enc}_U(d, x)) = (\text{OK}, \text{Eval}_g(d, x))$$

恰好在 $n(d)$ 次迭代后成立。如果识别边界还对 Safe 满足安全健全性, 那么这些轨迹也满足 Safe。

证明概要。 E4-D 初始化常量/输入前缀。在迭代 i 中, 描述符约束保证两个源都是较早的单元。E3 提供 q_0 , E2 从该类提供 g , 而 E4-D 仅将其写入规范目标, 同时保持已经建立的前缀不变。对前缀进行归纳, 可在 $n(d)$ 次迭代后得到 Eval_g ; 精确调度条款给出终止状态 OK。E1 将 E2 与 E4-D 使用的同一个功能位绑定到因果同后缀状态区分; 没有 E1, 归纳只能证明一个普通的有界 g 求值器, 而不是状态介导求值器。安全性仅由单独的健全性前提推出。□

该定理分解了局部门、重置、调度和帧义务; 它并未声称测试枚举了整个描述符域。它也没有证明 E4-A; 在 E4-A 中, 编译器会为每个电路生成不同的、被接受的 BPF 对象。

4. eBPF 校准案例

4.1 执行边界

程序 $P_U = \text{wm_circuit}$ 是一个固定的 eBPF 制品, 其节为 $\text{SEC}(\text{syscall})$ 。在所记录的环境中, 内核验证器接受该程序, 用户空间则通过 $\text{bpf_prog_test_run_opts}()$ 离线执行它; Linux 文档同时说明了用户空间程序执行以及 syscall 节/程序类型映射 [10]、[11]。其中不涉及任何实时钩子。该实验在本地以特权方式运行, 其接受与资源事实仍局限于所记录的程序类型、内核和体系结构。

载体边界如下:

宿主数据与验证	识别单元	接受后解释
文本 WMC1 $\rightarrow$ 规范化映射配置 $\text{Enc}_U(d, x)$	V 接受固定的 P_U , 而非 WMC1 或 d	P_U 读取映射 $\rightarrow \text{Sched}_U(d, x) \rightarrow$ 辅助函数返回值和导线观测

门映射 $G0$ 是一个专用的 BPF_MAP_TYPE_HASH 映射, 且 $\text{max_entries} = 2$ 。它不是 LRU 映射, 并保留默认预分配; Linux 文档说明了该映射类型的条目上限、默认预分配、更新标志, 以及成功/负错误值约定 [12]。该规约要求重置成功, 并要求一个外部临界区覆盖足迹中每个映射的设置、调用和读回。串行测试框架满足无交错使用模式, 但 eBPF 程序本身没有实现并发锁。

4.2 饱和秩 NAND

该门使用两两不同的键: 哨兵 S 、输入键 A 和输入键 B 。重置操作删除三者并插入 S , 从而建立占用数一。对于第一个输入, 零会更新 S , 一会插入新的 A ; 对于第二个输入, 零会更新 S , 一会插入新的 B 。命题 2 给出抽象占用规律。对于 Linux 实例, 已有键成功、低于容量时新键成功, 以及达到容量时新键失败, 均为仅限于有效参数、默认预分配、成功重置、无干扰及所记录 Linux 6.17 环境的显式前提。参考文献 [12] 提供映射接口和返回约定; 保留的原始返回值与机制控制在该对象上实例化了这一规律。该门只观测第二次更新的成功谓词, 而不观测可移植的错误编号。

四种执行如下:

a	b	重置后的操作 词	第二次操作前 的状态	第二次结果	输出 $[ret = 0]$
0	0	$updS; updS$	$\{S\}$	成功	1
0	1	$updS; insB$	$\{S\}$	成功	1
1	0	$insA; updS$	$\{S, A\}$	成功	1
1	1	$insA; insB$	$\{S, A\}$	失败	0

命题 2 (饱和秩 NAND)。令秩表示容量为 $k \geq 2$ 的资源中已占用名称的数量。重置在已有 S 的情况下建立秩 $k - 1$, 而两两不同的 A 和 B 是新名称。更新已有名称会成功且不改变秩; 当秩低于 k 时, 更新新名称会成功并增加秩; 当秩达到 k 时, 该操作会失败且不改变秩。将零分派给 S , 将第一个一分派给 A , 将第二个一分派给 B 。第二次更新的成功谓词为 $NAND(a, b)$ 。

*证明。*当 $a = 0$ 时, 第一次更新保持秩 $k - 1$, 因此无论哪一种第二次更新都会成功。当 $a = 1$ 时, 第一次更新将秩提高到 k ; 当 $b = 0$ 时, 由于 S 已存在, 第二次更新成功; 当 $b = 1$ 时, 由于 B 是新名称, 第二次更新失败。因此, 输出依次为 1, 1, 1, 0。□

NAND 具功能完备性, 因为 $\neg x = NAND(x, x)$, 且 $x \wedge y = \neg NAND(x, y)$ 。512 门界限电路规模, 不限制布尔基。

对于定义 1 的具体见证, 取 $P = wm_circuit$, 并令 ℓ 为 $circuit_step_cb$ 中第一次输入条件操作之后、第二次映射更新之前的点。令 $R_{G0}(\sigma)$ 为专用映射 $G0$ 的**完整辅助函数相关动态状态**, 包括键占用、桶、预分配元素/空闲链元数据, 以及更新辅助函数读取的任何其他映射局部状态; 取 $\rho_{obs}(\sigma) = R_{G0}(\sigma)$ 。已占用键集合 $K_{G0}(\sigma)$ 只是 $R_{G0}(\sigma)$ 的派生投影, 不能单独充当完整选定状态。

输入 $(0, 1)$ 和 $(1, 1)$ 在 ℓ 处分别到达 σ_0 与 σ_1 , 其派生占用集合为 $\{S\}$ 与 $\{S, A\}$, 故完整动态状态不同。公共剩余后缀 w 是插入新键 B , 随后统一捕获并存储成功位; 定义 $Obs(\tau) = [ret(\tau) = 0]$ 。切片和 ctx_w 包括共同程序点、键 B 、值、 $G0$ 身份与静态属性 (映射类型、容量、默认预分配和 BPF_ANY 模式)、相关控制与导线值, 以及位于 R_{G0} 外但被后缀读取的全部分量。**Env** 固定内核、对象、映射实例、调度和无干扰条件。

在声明的映射更新契约下, 两次后缀终止并分别观察到 1 和 0。较早输入不被后缀读取, 其持续且与后缀相关的效应包含于完整 R_{G0} 中, 所以非选定上下文相同。因而这是声明具体服务契约下的**条件性**定义 1 见证; 它比只选用集合更强, 也不声称保留轨迹暴露了全部内核字段。

在门规约下, 取 $s_{D_G}(\sigma) = (phase(\sigma), K(\sigma))$, 但只在由有效参数、专用预分配映射契约、成功重置、串行化和无干扰界定的不变区域 X_{D_G} 上使用。其操作充分性是显式服务契约前提, 不是对所有内核字段的机器检查模型。 $phase$ 取值于重置/哨兵/第一次操作/第二次操作阶段, 而 $K \subseteq \{S, A, B\}$ 且 $|K| \leq 2$ 。该载体及其未来观测商有限。完整重置加门词未必因果, 因为重置会消除传入差异; 见证由第一次操作后的内部前沿标记。取 $P_G = P_U$ 、 $Adm_G = X_{D_G}$, 上述见证支持 E1, 命题 2 与重置前提支持 E2–E3。

该机制并未为 eBPF 增加外延意义上的布尔表达能力。它是一个校准案例, 表明有文档记录的有状态服务可以提供提供一个可复用的接受后阶段逻辑门; 它本身并不能确立报告遗漏。

4.3 固定解释器与映射 ABI

除 $G0$ 外, $wm_circuit$ 还使用 $TAPE$ 和四个解释器映射:

- $TAPE$ 记录构建变体、容量、选定的原始返回值和错误计数;
- $CIRCUIT$ 存储规范化记录 $(op, src0, src1, dst)$;
- $WIRES$ 存储常量、主输入和规范的 SSA 风格门输出;

- `VM_CONTROL` 提供 ABI 版本和声明的计数；
- `VM_TRACE` 存储每个门的有效性与输出，并且对于状态介导变体，还存储第二个辅助函数的原始返回值。

宿主解析器检查并规范化 WMC1、写入映射单元，并保留所请求的输出列表。只有在 `status = OK` 后才会投影该列表，且 BPF 从不读取该列表。源代码层面的掩蔽由宿主控制流保护；负向测试套件测试拒绝状态和执行计数，而非注入的运行时掩蔽路径。在内核内部，一个有界循环处理至多 512 个描述符。每次迭代在调用辅助函数之前复制描述符和源值，重置 `G0`，调用该门，并按键更新规范目标，从而避免在这些调用之间保留映射值指针。

规范目标防止源/目标别名和前向引用。所覆盖的格式错误版本、计数、操作码、目标或源会得到非 `OK` 状态。对于有效描述符，拓扑限制保证每个源都位于更早位置。外部的全事务串行化防止并发混合配置；连续的串行调用会覆盖共享单元，而过期的物理单元可能仍然保留。

验证器的接受单元是固定的解释器制品，而非每个描述符，尽管测试框架可能针对不同数据集重新加载同一个对象。改变映射配置会改变被解释的电路。这是 E4-D 的载体区分，而不是编译器为每个电路生成新的被接受 BPF 对象的证据。

4.4 源代码层面的不变式与证据边界

引理 1 (源代码层面的解释器前缀不变式)。假设命题 2 的更新律、成功重置和必需的映射/辅助函数调用、E4-D 串行化环境、有效的 $\text{Enc}_U(d, x)$ ，并且所捕获的转换后对象保留了经人工检查的源代码控制/数据依赖。对于每个满足 $0 \leq t \leq n(d)$ 的整数 t ，在 t 次成功回调之后，索引低于 $2 + m(d) + t$ 的规范导线所构成的序列等于 $\text{Eval}_{\text{NAND}}(d, x)$ 的对应前缀。

证明概要。外层程序初始化常量，并保留宿主写入的输入。回调验证当前操作码、目标和较早的源，随后复制描述符和源比特。命题 2 给出重置后的 NAND；成功时仅写入规范目标，并增加完成计数。外层程序通过 `bpf_loop` 请求 $n(d)$ 次迭代；辅助函数契约规定回调返回 0 继续、返回 1 停止，并报告实际迭代次数 [13]。解释器只有在该返回值与自身完成计数都等于 $n(d)$ 时才接受。对 t 进行归纳即可得到前缀结论。这是由保留的转换后转储支持人工检查的源代码层面论证，而非经过机器检查的 eBPF 语义证明。□

E4-D 实现论证与制品的对应关系如下：

条款	源代码/捕获证据	实证检查或剩余前提
1 — 固定接受单元	保留的正常对象、验证器元数据、捕获的已加载程序标签	描述符变化不会产生新的 BPF 制品
2 — 初始状态与精确调度	<code>wm_circuit</code> 初始化常量、调用 <code>bpf_loop</code> ，并检查循环返回值/完成计数	命名、随机、零、深度和联合边界运行
3 — 串行化	运行器串行调用各项配置；程序不含锁	覆盖整个映射集合的临界区仍是外部前提
4 — 门/帧条件	<code>circuit_step_cb</code> 验证/复制源、重置 <code>G0</code> ，并更新一个规范目标	语义审计器检查所发出的目标与输出
5 — 边界/错误	计数防护、回调失败状态、宿主侧状态掩蔽防护	八个负向案例检查状态/计数；掩蔽路径仅由源代码防护

因此，A 已记录；C 只在声明的具体服务/无干扰契约下有条件性见证；P 在额外的源到对象、帧保持及串行化前提下得到支持。节点 R 和 W 缺失：未提供 Linux 报告单元提取器或部署策略。

5. 评估

5.1 记录的环境与主要运行

主要证据是包含源码快照的运行证据包 *results/interpreter/interpreter-final-20260711-02/*。它记录了 Ubuntu 24.04、aarch64 上的 Linux 6.17.0-35-generic，并保存了四个 BPF 变体、所捕获的已加载程序元数据、验证器日志、描述符、JSONL 输出、由作者执行的语义审计、当时选定的源码/手稿快照，以及一份自行签发的 SHA-256 清单。最终手稿晚于该运行，不受该清单覆盖。这是一次由作者生成的运行，并带有一项单独由作者执行的审计，而非第三方复现。

该数据集恰好包含

$38,533 = 26,488 \text{ 条逐门记录} + 12,037 \text{ 条成功运行记录} + 8 \text{ 条负对照记录}.$

这些是异构的证据行，而不是“38,533 个测试”。特别是，显式算术基线的门记录不包含由状态介导的辅助函数返回值轨迹。

5.2 覆盖范围

数据集	输入覆盖	成功运行次数	逐门记录数	目的
9 个具名电路	对每个电路穷举	39	166	典型组合函数，包括 NAND、多路复用器和加法器
100 个随机 DAG	对每个 DAG 穷举；至多 6 个输入和 24 个门；种子 3235823838	1,876	23,776	固定种子的结构与语义回归测试
深度边界	对一条 512 门链的单个输入取两种赋值	2	1,024	门数量与深度边界
联合边界	全零向量和全一向量	2	1,024	64 个输入、512 个门、578 条导线，包括导线 577
零门边界	对 0 输入/0 门常量描述符的专门重复测试	1	0	空电路执行
串行交替	NAND/全加器/多路复用器交替	10,000	0	重置与跨调用污染回归测试
容量 64 对照	具名电路的输入	39	166	消除容量饱和
强制哨兵对照	具名电路的输入	39	166	消除第二次新键插入
显式基线	具名电路的输入	39	166	普通算术 NAND 的接受性与语义
畸形核心	8 个 ABI/计数/操作/目标/引用案例	—	—	预期得到非 OK 状态与执行计数

每个具名电路和随机电路都穷举其自身的有限输入域。64 输入联合边界必然使用两个选定向量，而不是全部 2^{64} 个输入。因此，该语料库是针对实现的回归证据，也是对证明前提的检查；它既不是对所有 $D_{64,512}$ 描述符的枚举，也不是实证证明。

串行交替数据集复用同一个已加载的测试框架，连续调用 10,000 次。它可检测串行使用下常见的重置或陈旧状态回归。它不是并发压力测试，因而无法确立全局临界区前提；该前提仍位于 eBPF 程序之外。

5.3 单独的语义审计与完整性检查

由作者执行的语义审计器不信任运行器的通过标志。其另一套实现：

1. 重构每一个具名源描述符以及两个生成的边界描述符；
2. 根据固定种子逐字节重新生成 100-DAG 语料库；
3. 解码 WMC1，重新计算预期的完整线向量，并比较每个发出的门目标值和投影输出；
4. 检查每条边界门记录以及全部八个畸形核心案例；
5. 将每个 JSONL 运行时标签与所捕获的已加载程序元数据进行交叉核对。

语义审计报告成功。完成该审计后，运行脚本写入并验证一份自行签发的 SHA-256 清单，作为单独的完整性检查；其验证器同样报告成功。该清单能检测证据包的意外偏离，但不构成强来源证明、签名、时间戳、认证或独立复现。所捕获的标签是一项防混淆检查，而源代码快照仅覆盖测试框架选定的文件。

5.4 机制归因

三个变体将容量机制与真值表及制品接受性区分开来。

第一，将容量从 2 增加到 64，可消除被测调度中的饱和；具名语料库对照中的全部 166 个门输出均变为 1。第二，强制第二个一位输入更新 S ，可消除新 B 的插入；全部 166 个输出同样变为 1。第三，一个算术基线在不使用容量机制的情况下计算 NAND，并产生预期的具名电路结果。全部四个变体——状态介导的门、两个对照以及基线——都能在记录的环境中加载并执行。

这些对照支持机制归因：饱和和第二个新名称对于门输出零都是必要的。它们并未显示等价的验证器报告。基线确认，该机制没有获得普通字节码无法实现的新布尔函数。

5.5 校准结果

Linux 的验证器文档将其目标表述为判定程序安全性并验证路径、参数和内存访问 [14]；它并未规定对由映射介导的计算给出完备的功能证书。没有 stock-Linux 报告提取器时，本文既不测试也不推断它跟踪辅助函数返回值的精度。本案例是一项校准，而非不健全接受的证据：它在声明的服务契约下给出条件性 C 见证，并在额外实现前提下支持节点 P；该 stock-Linux 校准的 R 仍未建立，离线案例也没有建立 W。

5.6 固定辅助可执行报告实例

为在不重命名 Linux 验证器制品的前提下执行报告相对判据，将辅助 R 实例嵌入第 7.1 节的完整模型载体并固定

$$M_{linux_r_aux_v1} = (V_{linux_r}, I_{hash}, Report_{aux}, K_{obs}, P_{aux}, \ell, D, F, \emptyset, \emptyset, \emptyset, \emptyset).$$

P_{aux} 由自定义报告生成识别器 V_{linux_r} 接受；它不是经 stock Linux 验证器接受的 BPF 对象。 I_{hash} 是有限、确定、串行且无干扰的受限语义，只覆盖固定程序使用的非驱逐 HASH 映射更新情形。这里 $Report_{aux}$ 只指 `report.json["report_cells"]`；`derivation.json` 仅记录工作列表/计算溯源，不属于报告标签接口。最

后四项分别是有类型的空行为主体集、空效果集、空驱动关系和空许可效果集；它们只把 R 实例嵌入共同载体，不提出 W 论断。

可执行证书。 $R(M_{linux_r_aux_v1})$ 成立（制品状态：established）。

证书论证。识别器接受 P_{aux} 在单一串行无干扰环境中穷尽 $a \in \{0, 1\}, b = 1$, 得到 $F = \text{Reach}_{I_{hash}}(P_{aux}, \ell) = \{\text{frontier:S}, \text{frontier:AS}\} \subseteq X_D$; X_D 还包含相应的两个终止状态。令 $a_{\text{obs}} \in A_D$ 表示名为 `update-suffix-and-observe` 的规约动作, 以 c_{obs} 表示精确编码的运行时的操作符号 `bpf_map_update_elem(G0, suffix_key, one, BPF_ANY); observe(ret==0)`, 并令 $\iota_{P_{aux}, \ell}(a_{\text{obs}}) = c_{\text{obs}}$; 这个单元素编码是单射。 $\iota_{P_{aux}, \ell}(a_{\text{obs}})$ 的具体执行在两个前沿状态上有定义, 在终止状态上无定义, 并在定义性、输出和后继上与 D 一致, 故检查器在整个 X_D 上建立操作充分性。观测契约把 ρ_{obs} 固定为已占用键集合, 把 Obs 固定为有序成功位词, 把 Slice 固定为程序阶段/服务上下文对, 把 Env 固定为上述单一环境; 特别地, $\rho_{\text{obs}}(\text{frontier:S}) = \{S\} \neq \{S, A\} = \rho_{\text{obs}}(\text{frontier:AS})$ 。对于 F^2 中每个有序状态对和 $\mathcal{W}_D(F) = \{\varepsilon, a_{\text{obs}}\}$ 中每个词, 检查器均建立其余运行时词、共同上下文、观察者兼容性与 K_{obs} 健全性义务。这些义务履行了除唯一单元覆盖之外的全部可接纳条件。

Report_{aux} 发出一个唯一单元 $a^\#$, 共同覆盖两个前沿状态, 从而完成 $\text{Adm}(P_{aux}, \ell, D, F; K_{\text{obs}})$ 。精确有限未来观测商把二者分到不同类, 故 $|\beta_D(F_{a^\#})| = 2$, 且 a_{obs} 分别输出 1 与 0。同一后缀因此先见证实定义 1, 而带标签元组 $(P_{aux}, \ell, D, F, a^\#, a_{\text{obs}})$ 满足定义 2, 所以 $R(M_{linux_r_aux_v1})$ 成立。证据另含 21 项域/动作的**返回类别包含检查**和 2 项报告单元的**后继包含检查**, 均为零违反; 前 21 项不是 21 个完整后续状态检查。精确占用跟踪、容量 64、强制哨兵、忽略返回四个负对照均消除 R 。□

独立可执行检查器在不导入模型实现的情况下重构可达性、唯一单元覆盖、商和因子分解判定。归档的 VM 运行与审计均通过; 单元测试会构造两个固定输入证据包并逐字节比较其形式化 JSON。这是可执行有限模型证据, 不是机器检查证明。保留的内核校准对 $(0, 1)$ 与 $(1, 1)$ 两个赋值共有 4 行 oracle; 它只校准受限服务情形, 不证明 I_{hash} 与 Linux 之间的 refinement 或 bisimulation, 也不提取 stock 验证器单元。因此 stock Linux 的 R 、 W 、 WM_{shape} 以及任何普遍必要性猜想仍未建立。证据路径为 `results/linux_r/linux-r-v1/`。

5.7 有效性威胁

实验使用了一个内核构建版本和一种体系结构。论证依赖于一个专用的预分配非 LRU 映射、成功重置、互异的键、所述更新律, 以及整个映射集合上的互斥。会发出门记录的数据集保留原始返回值; 10,000 次运行的压力测试文件记录运行/状态/输出证据, 但不包含逐门原始行。证明只使用成功与失败之分, 并不承诺可移植的错误码。

有限测试不能证明所有 $D_{64,512}$ 。相反, 定理 1 依赖于所陈述的源代码级初始化、验证、有序迭代、门和帧义务; 语料库旨在寻找反例。转换后的转储和验证器日志被保留下来以供人工检查及清单哈希使用, 而未被自动化数据流检查器消费。状态掩蔽具有源代码顺序保护, 而负面测试套件测试的是拒绝/状态, 并非注入的掩蔽路径。不存在机器检查的 eBPF 语义证明, 且只有在可选的 **Safe** 结论中才假定生产验证器的安全健全性。

6. 相关工作

语言理论安全提供识别/解释框架 [1]–[4]; 对象中心追踪重建状态依赖的底层操作语言 [15]。怪异机器工作研究非预期计算与可利用性 [16], [17], 不安全编译明确策略边界 [18], 携带证明代码工作识别证明模型之外的影子执行 [19]。合法 RPM 元数据也可产生计算, 漏洞流类型可导出抽象怪异机器 [20], [21]; 这些工作没有采用本文针对一个已接受制品的实际报告单元判据。

抽象解释提供具体/抽象、健全性和完备性词汇 [5], [22]; 本文只讨论一个前沿上唯一分配的实际计算单元。MOAT 隔离接受后的 BPF 效果 [23], mismorphism 比较不同解释 [24]。eBPF 范围分析验证与状态嵌入针对验证器逻辑的健全性或覆盖错误 [25], [26]。VEP 的“programmability”是减少验证工具链限制; Rex 则通过语言级安全与语言外运行时替代独立静态验证器, 处理拒绝侧的语言-验证器失配 [27], [28]。本文 P 不同: 它要

求一个经原生验证器接受的固定制品解释有界族。DRACO 在验证器接受后检查功能规范；BPFVERIFY 把 eBPF 字节码翻译为位与内存精确的 Horn 子句以进行功能验证 [29], [30]。

2026 年预印本进一步界定边界：Heimdall 处理编译与接受后仍存的高层缺陷；Yaksha-Prashna 提取第三方 eBPF 字节码行为；bpfix 定位被拒程序中的证明丢失 [31]–[33]。信任边界语义缝隙工作分析正确语法接受后仍不足的断言 [34]。这些工作没有联合区分本文图谱的全部义务；反过来，本文 stock-Linux 校准案例也未建立 R 或 W，而独立的辅助元组只建立其自身的 R。四项 2026 工作均按预印本引用，而非声称已经同行评审。

7. 局限、启示与展望

7.1 论断图谱具有严格区分

为将全部五个节点置于同一载体上，固定模型 $M = (V, I, \text{Report}, K_{\text{obs}}, P_*, \ell_*, D_R, F_R, \mathcal{A}, \mathcal{E}, \text{Drive}, \text{Pol})$ ，并附加具名的预期语义、安全、报告抽象和粒度契约。 $\mathcal{A}$ 与 $\mathcal{E}$ 是行为主体和效果集合，Drive 记录主体可通过指定 P_* 驱动的效果， $\text{Pol} \subseteq \mathcal{E}$ 是允许效果集。

定义： $A(M)$ 当 $P_* \in L_V$ ； $C(M)$ 当存在 $(P_*, \ell, w) \in L_{\text{causal}}$ ； $P(M)$ 当 P_* 是同一个固定已接受有界解释器，其同制品门基满足 E1–E3， P_* 用该基履行 E4-D，且 g 与常量 $\{0, 1\}$ 构成功能完备布尔基； $R(M)$ 当 $\text{Adm}(P_*, \ell_*, D_R, F_R; K_{\text{obs}})$ 成立且某个 $(P_*, \ell_*, D_R, F_R, a^\#, w)$ 恰在该元组上为输出见证残差； $W(M)$ 当存在主体驱动 $e \notin \text{Pol}$ 。

令 $\text{Link}(M)$ 要求 R 见证属于建立 P 的门/解释器执行族，且 W 效果来自主体驱动同一编码计算； $\text{Unint}(M)$ 表示该解释/效果在预期语义之外。定义

$$\text{WM}_{\text{policy}}(M) = P(M) \wedge W(M) \wedge \text{Link}(M) \wedge \text{Unint}(M).$$

令 $\text{Doc}(M)$ 表示链接行为使用已文档化语义且维持声明安全契约， $\text{Conf}(M)$ 表示报告实现符合指定抽象， $\text{Gran}(M)$ 表示 R 碰撞由声明报告粒度强制。更窄的分类为

$$\text{WM}_{\text{shape}}(M) = \text{WM}_{\text{policy}}(M) \wedge R(M) \wedge \text{Doc}(M) \wedge \text{Conf}(M) \wedge \text{Gran}(M).$$

命题 3 (论断节点之间的非蕴含关系)。在此模型类上，下列蕴含均不成立：

$$A \not\Rightarrow C, \quad C \not\Rightarrow P, \quad C \not\Rightarrow R, \quad P \not\Rightarrow R, \quad R \not\Rightarrow P, \quad P \not\Rightarrow W, \quad R \not\Rightarrow W.$$

定义直接给出 $R \Rightarrow C \Rightarrow A$ 与 $P \Rightarrow C \Rightarrow A$ ：定义 2 内含定义 1 见证，而 E1 使用 E4-D 的同一个制品。策略层面的怪异机器不一定满足 R；更窄的形状分类必须满足上式中的 Link、Unint、Doc、Conf 与 Gran，不能简化为裸 $P \wedge R \wedge W$ 。

由有限反模型给出的证明。对于 $A \not\Rightarrow C$ ，仅接受 *skip* 并拒绝另一个安全的常量程序；该被接受系统不存在因果操作。对于 $C \not\Rightarrow P$ 和 $R \not\Rightarrow P$ ，使用状态 $\{0, 1, \perp\}$ 、一次输出该位并转移至汇点 $\perp$ 的读取操作、不设重置，并使用一个同时覆盖 0 和 1 的报告单元。对于 $C \not\Rightarrow R$ ，使用一个可重置的双状态转导器，其报告对每个未来观测类各有一个单元。对于 $P \not\Rightarrow R$ ，使用任意满足 E1–E3 和 E4-D 的、被接受的有界 NAND 解释器，并采用相同的精确报告划分。对于 $P \not\Rightarrow W$ ，令 $\text{Pol} = \mathcal{E}$ ；采用具有同样宽松策略的单报告单元破坏性读取模型，也可得到 $R \not\Rightarrow W$ 。最后，选择一个满足 P 的模型，使主体从同一编码 P 执行族驱动一个非预期且被策略排除的效果，并令报告精确区分全部相关未来观测类；于是 $\text{WM}_{\text{policy}} \wedge \neg R$ 。因此，策略层面的怪异机器分类一般不要求 R。□

定理 1 仅处理在给定显式重置与组合前提后从 C 到 P 的构造。裸 C、标准抽象解释的不完备性或接受不完备

性，都不蕴含 P 或怪异机器分类。

7.2 防御启示

该模型提出一种三部分审计。分别声明被识别的属性和报告；枚举已接受代码可以驱动的有状态操作语言，包括环境假设；随后测试与安全相关的商类是否可通过所选报告进行因子分解。违反契约需要修复实现。符合文档的行为若违反预期的报告关系，则需要细化报告、限制运行时或进行隔离。如果报告从未承诺该关系，且策略允许该行为，那么可能不存在验证器缺陷。

7.3 怪异机器地位与未来的形状定理

在源到对象对应关系与串行化前提下，stock eBPF 案例支持节点 P：被接受的代码控制、重置并组合一个有状态 NAND 基底。它并未确立 R 或 W。辅助实例 $M_{linux_r_aux_v1}$ 只针对固定自定义报告和受限服务语义建立 R；它不建立 W 或 WM_{shape} ，也不补足缺失的 stock 报告嵌入。即使得到一个已文档化碰撞，也还须证明报告实现符合其声明抽象且碰撞由报告粒度强制；当前 Linux 制品未提供这些证据。怪异机器分类还要求行为主体、同一编码计算上的链接、非预期性和被策略排除的效果，而离线运行没有建立这些义务。

未来的形状定理必须推导而非假定两个闭包性质。**宏闭包**必须在预算内重命名并组合局部操作时保持接受性。**报告嵌入**必须在组合之后保留实际已计算单元中的相关碰撞。上下文敏感分析可以使任一性质失效，且预算不一定具有可加组合性。以声明的观测语义实例化的完备壳理论 [22]，或许可以刻画所需的报告细化，但它不会凭空产生可达性、控制、重置、路由、同计算链接或策略违反。

7.4 其余局限

证据是在一个 Linux/aarch64 设置上的一个有界组合电路解释器。它不是第二套系统的结果、并发结果、E4-A 编译器结果或无界结果。由于不存在 Linux 报告提取器，报告框架与校准案例在节点 R 处仍然断开。最终证据包由作者生成，并带有一项单独由作者执行的语义审计；它不是第三方复现。

8. 结论

程序验证是一道语言理论安全边界，但制品接受与接受后解释使用不同的语言。我们将以已接受制品为索引的因果族 L_{causal} 与相对于报告的 L_{res}^R 分离，给出了行为因子分解判据，并分解了有界状态介导组合的各项义务。

stock eBPF 校准记录 A，在声明的具体服务/无干扰契约下条件性见证 C，并在显式的源到对象、环境和帧条件前提下支持 P。一个双条目映射的饱和更新代数实现了 NAND，而一个固定制品可处理至多具有 64 个输入、512 个门和 578 条导线的电路。它并未确立 Linux 报告的不可因子分解性，也未确立策略/威胁义务。另一个固定辅助元组通过一个实际计算单元碰撞建立 R，却不建立 W、 WM_{shape} 、stock Linux 报告结论或普遍必要性定理。

伦理与数据可用性

所有实验均在隔离的本地虚拟机中运行，不将任何程序附加到实际运行中的网络或内核钩子，不以任何第三方为目标，也不尝试内存破坏、验证器绕过或权限提升。该制品仅使用合法的辅助函数调用和有界执行。

配套仓库位于 <https://github.com/Emtanling/eBPF-machine>。不可变证据位于提交 4309069a 下的 `results/interpreter/interpreter-final-20260711-02/`；辅助报告实例证据位于提交 f665b1a 下的

`results/linux_r/linux-r-v1/`。两个证据包分别包含相关模型或程序输入、输出、审计材料及自行签发的完整性清单；清单只防止意外漂移，不构成独立来源证明。

致谢与 AI 使用声明

OpenAI Codex 协助起草、修订、语言编辑、制品代码/测试修订以及作者指导的检查。作者独立审查主张、证明、引用、更改与结果并承担全部责任。作者声明不存在利益冲突。

参考文献

- [1] L. Sassaman, M. L. Patterson, S. Bratus, and M. E. Locasto, “Security Applications of Formal Language Theory,” *IEEE Systems Journal*, vol. 7, no. 3, pp. 489–500, 2013, doi: 10.1109/JSYST.2012.2222000.
- [2] F. Momot, S. Bratus, S. M. Hallberg, and M. L. Patterson, “The Seven Turrets of Babel: A Taxonomy of LangSec Errors and How to Expunge Them,” in *2016 IEEE Cybersecurity Development (SecDev)*, pp. 45–52, 2016, doi: 10.1109/SecDev.2016.019.
- [3] L. Sassaman, M. L. Patterson, and S. Bratus, “A Patch for Postel’s Robustness Principle,” *IEEE Security & Privacy*, vol. 10, no. 2, pp. 87–91, 2012, doi: 10.1109/MSP.2012.31.
- [4] S. Ali, P. Anantharaman, Z. Lucas, and S. W. Smith, “What We Have Here Is Failure to Validate: Summer of LangSec,” *IEEE Security & Privacy*, vol. 19, no. 3, pp. 17–23, 2021, doi: 10.1109/MSEC.2021.3059167.
- [5] P. Cousot and R. Cousot, “Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints,” in *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL ’77)*, pp. 238–252, 1977, doi: 10.1145/512950.512973.
- [6] A. Nerode, “Linear Automaton Transformations,” *Proceedings of the American Mathematical Society*, vol. 9, no. 4, pp. 541–544, 1958, doi: 10.1090/S0002-9939-1958-0135681-9.
- [7] G. H. Mealy, “A Method for Synthesizing Sequential Circuits,” *Bell System Technical Journal*, vol. 34, no. 5, pp. 1045–1079, 1955, doi: 10.1002/j.1538-7305.1955.tb03788.x.
- [8] G. Amato and F. Scozzari, “Observational Completeness on Abstract Interpretation,” *Fundamenta Informaticae*, vol. 106, nos. 2–4, pp. 149–173, 2011, doi: 10.3233/FI-2011-381.
- [9] F. Ranzato and F. Tapparo, “Generalized Strong Preservation by Abstract Interpretation,” *Journal of Logic and Computation*, vol. 17, no. 1, pp. 157–197, 2007, doi: 10.1093/logcom/exl035.
- [10] Linux Kernel Documentation, “Running BPF Programs from Userspace,” [Online]. Available: Linux v6.17 BPF documentation (accessed Jul. 11, 2026).
- [11] Linux Kernel Documentation, “Program Types and ELF Sections,” [Online]. Available: Linux v6.17 libbpf documentation (accessed Jul. 11, 2026).
- [12] Linux Kernel Documentation, “BPF_MAP_TYPE_HASH, with PERCPU and LRU Variants,” [Online]. Available: Linux v6.17 map documentation (accessed Jul. 11, 2026).

- [13] Linux Kernel Developers, “bpf_loop Helper API,” *include/uapi/linux/bpf.h*, Linux v6.17. [Online]. Available: Linux v6.17 source (accessed Jul. 13, 2026).
- [14] Linux Kernel Documentation, “eBPF Verifier,” [Online]. Available: Linux v6.17 verifier documentation (accessed Jul. 11, 2026).
- [15] I. Palmer, E. Rogers, and R. Adams, “Object-Centric Tracing for Language-Theoretic Security in Low-Level Interfaces,” in *Twelfth Workshop on Language-Theoretic Security (LangSec 2026)*, 2026. [Online]. Available: langsec.org (accessed Jul. 12, 2026).
- [16] S. Bratus, M. E. Locasto, M. L. Patterson, L. Sassaman, and A. Shubina, “Exploit Programming: From Buffer Overflows to Weird Machines and Theory of Computation,” *USENIX ;login:*, vol. 36, no. 6, pp. 13–21, 2011.
- [17] T. Dullien, “Weird Machines, Exploitability, and Provable Unexploitability,” *IEEE Transactions on Emerging Topics in Computing*, vol. 8, no. 2, pp. 391–403, 2020, doi: 10.1109/TETC.2017.2785299.
- [18] J. Paykin et al., “Weird Machines as Insecure Compilation,” arXiv:1911.00157, 2019, doi: 10.48550/arXiv.1911.00157.
- [19] J. Vanegue, “The Weird Machines in Proof-Carrying Code,” in *2014 IEEE Security and Privacy Workshops*, pp. 209–213, 2014, doi: 10.1109/SPW.2014.37.
- [20] S. Ali, M. E. Locasto, and S. W. Smith, “Weird Machines in Package Managers: A Case Study of Input Language Complexity and Emergent Execution in Software Systems,” in *2024 IEEE Security and Privacy Workshops (SPW)*, pp. 169–179, 2024, doi: 10.1109/SPW63631.2024.00021.
- [21] M. Lesani, “Vulnerability Flow Type Systems,” in *2024 IEEE Security and Privacy Workshops (SPW)*, pp. 157–168, 2024, doi: 10.1109/SPW63631.2024.00020.
- [22] R. Giacobazzi, F. Ranzato, and F. Scozzari, “Making Abstract Interpretations Complete,” *Journal of the ACM*, vol. 47, no. 2, pp. 361–416, 2000, doi: 10.1145/333979.333989.
- [23] H. Lu, S. Wang, Y. Wu, W. He, and F. Zhang, “MOAT: Towards Safe BPF Kernel Extension,” in *33rd USENIX Security Symposium*, pp. 1153–1170, 2024.
- [24] P. Anantharaman et al., “Mismorphism: The Heart of the Weird Machine,” in *Security Protocols XXVII*, LNCS 12287, pp. 113–124, 2020, doi: 10.1007/978-3-030-57043-9_11.
- [25] H. Vishwanathan, M. Shachnai, S. Narayana, and S. Nagarakatte, “Verifying the Verifier: eBPF Range Analysis Verification,” in *Computer Aided Verification*, Lecture Notes in Computer Science, vol. 13966, pp. 226–251, 2023, doi: 10.1007/978-3-031-37709-9_12.
- [26] H. Sun and Z. Su, “Validating the eBPF Verifier via State Embedding,” in *18th USENIX Symposium on Operating Systems Design and Implementation*, pp. 615–628, 2024.
- [27] X. Wu et al., “VEP: A Two-stage Verification Toolchain for Full eBPF Programmability,” in *22nd USENIX Symposium on Networked Systems Design and Implementation*, pp. 277–299, 2025.
- [28] J. Jia et al., “Rex: Closing the Language-Verifier Gap with Safe and Usable Kernel Extensions,” in *2025 USENIX Annual Technical Conference*, pp. 325–342, 2025.
- [29] D. Lu, B. Tang, M. Paper, and M. Kogias, “Towards Functional Verification of eBPF Programs,” in *Proceedings of the 2024 ACM SIGCOMM Workshop on eBPF and Kernel Extensions (eBPF ’24)*, pp. 37–43, 2024, doi: 10.1145/3672197.3673435.

- [30] M. Bromberger, S. Schwarz, and C. Weidenbach, “Automatic Bit- and Memory-Precise Verification of eBPF Code,” in *Proceedings of the 25th Conference on Logic for Programming, Artificial Intelligence and Reasoning*, EPIc Series in Computing, vol. 100, pp. 198–221, 2024, doi: 10.29007/sj4l.
- [31] V. A. Dasu, M. Santra, M. R. U. Rashid, A. Kumar, S. Tizpaz-Niari, and G. Tan, “Heimdall: Formally Verified Automated Migration of Legacy eBPF Programs to Rust,” arXiv:2605.25411, 2026, doi: 10.48550/arXiv.2605.25411.
- [32] A. Singh et al., “Yaksha-Prashna: Understanding eBPF Bytecode Network Function Behavior,” arXiv:2602.11232, 2026, doi: 10.48550/arXiv.2602.11232.
- [33] Y. Zheng et al., “Characterizing and Bridging the Diagnostic Gap in eBPF Verifier Rejections,” arXiv:2607.02748, 2026, doi: 10.48550/arXiv.2607.02748.
- [34] D. Kim, J.-Y. Choi, and J. Lee, “Trust Boundary Semantic Gaps: A Multi-dimensional Analysis and Mitigation for Security-by-Design,” arXiv:2607.01711, 2026, doi: 10.48550/arXiv.2607.01711.